

Final Project Report

Project Title: Secure Chat Application with SSL Encryption

by Huynh Khoa Le | for Network Programming Course

1. Abstract

This project develops a Secure Chat Application using Python that enables users to exchange real-time messages through an encrypted connection. The application follows a client-server architecture using TCP sockets and secures communication with the SSL/TLS protocol.

A graphical user interface (GUI) was implemented using Tkinter, and all conversations are automatically stored in a log file. The project demonstrates practical knowledge of socket programming, multithreading, SSL encryption, and GUI development while meeting the project requirements for secure communication.

2. Introduction

Online chat applications are widely used for communication, making data security an important concern. Messages transmitted without encryption may be intercepted or accessed by unauthorized users. Therefore, secure communication protocols are essential for protecting user privacy.

The purpose of this project is to develop a Secure Chat Application using Python that allows multiple users to exchange real-time messages through an encrypted SSL/TLS connection. The application also records conversation history in a log file and provides a simple graphical user interface for user interaction.

Through this project, key networking concepts such as socket programming, multithreading, and secure communication are applied in a practical application.

3. Related Study

- **Python Socket Programming**

Python's socket module provides low-level networking functionality that enables communication between computers using TCP/IP. Socket programming forms the foundation of the client-server communication model implemented in this project.

- **TCP Client-Server Architecture**

The application follows a client-server architecture where the server continuously listens for incoming client connections. Each connected client exchanges messages through the central server, allowing multiple users to communicate simultaneously.

- **SSL/TLS Encryption**

SSL (Secure Sockets Layer) and its successor TLS (Transport Layer Security) provide encrypted communication channels over TCP connections. Python's ssl module wraps ordinary sockets into secure sockets, automatically encrypting and decrypting transmitted data.

- **Python Cryptography Library**

The cryptography package was used only during project setup to generate a self-signed SSL certificate and private key. These certificates are later loaded by the SSL module whenever the server starts.

- **Python Threading**

Python's threading module allows the server to handle multiple clients concurrently. Each client connection runs in its own thread, allowing real-time communication without blocking other users.

- **Tkinter GUI**

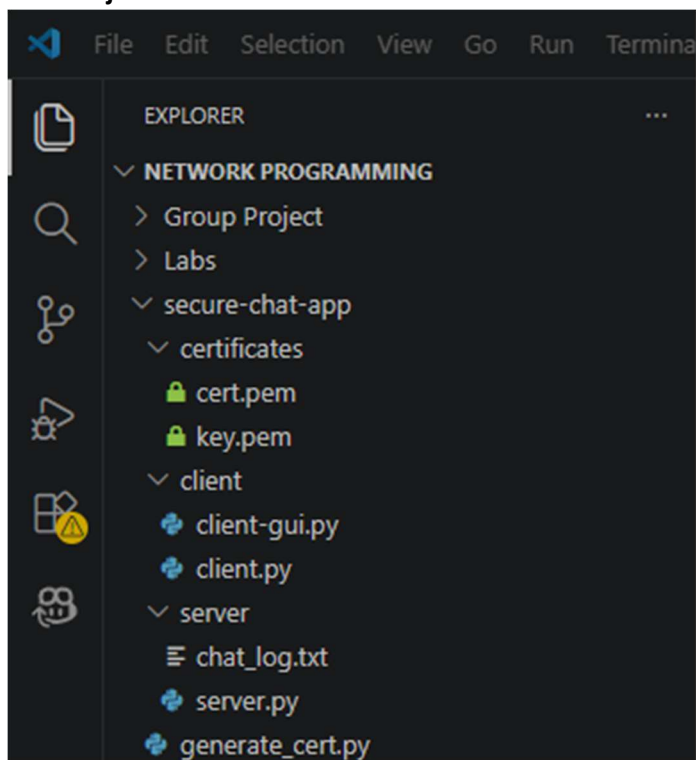
Tkinter is Python's standard graphical user interface library. It was used to create the chat window, username input, message area, and send button, providing an intuitive interface for users.

4. Application Design

- **Development Environment**

Component	Version / Description
Programming Language	Python 3.13.7
IDE	Visual Studio Code
Operating System	Windows 11
GUI Library	Tkinter
Networking	Python Socket Module
Security	Python SSL Module
Certificate Generator	Cryptography Package
Threading	Python Threading Module

- **Project Structure**



- **Main Components**

Server (server.py)	The server listens for incoming client connections, performs SSL encryption, broadcasts incoming messages to connected clients, and stores all conversations in a log file.
Client (client.py)	The client establishes an encrypted connection to the server using SSL, sends messages entered by the user, and continuously receives incoming messages from other connected users.
GUI Client (client-gui.py)	The graphical interface allows users to enter their username, connect to the server, send messages, and view the chat history through an interactive window.

- **Packages Used**

Package	Purpose
socket	Network communication
ssl	Secure encrypted communication
threading	Handle multiple clients simultaneously
tkinter	Build graphical user interface
datetime	Generate timestamps
cryptography	Generate SSL certificates

- **Data Storage**

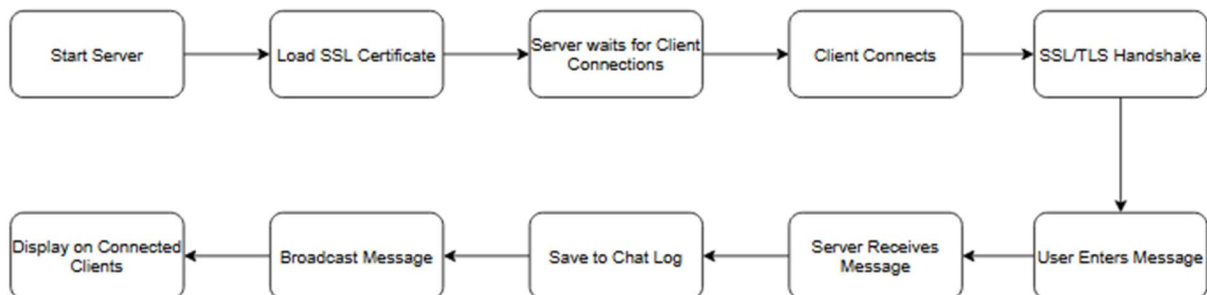
All conversations are automatically stored in the file: *server/chat_log.txt*.

Each message contains a timestamp and the sender's username for future reference.

- **Security Implementation**

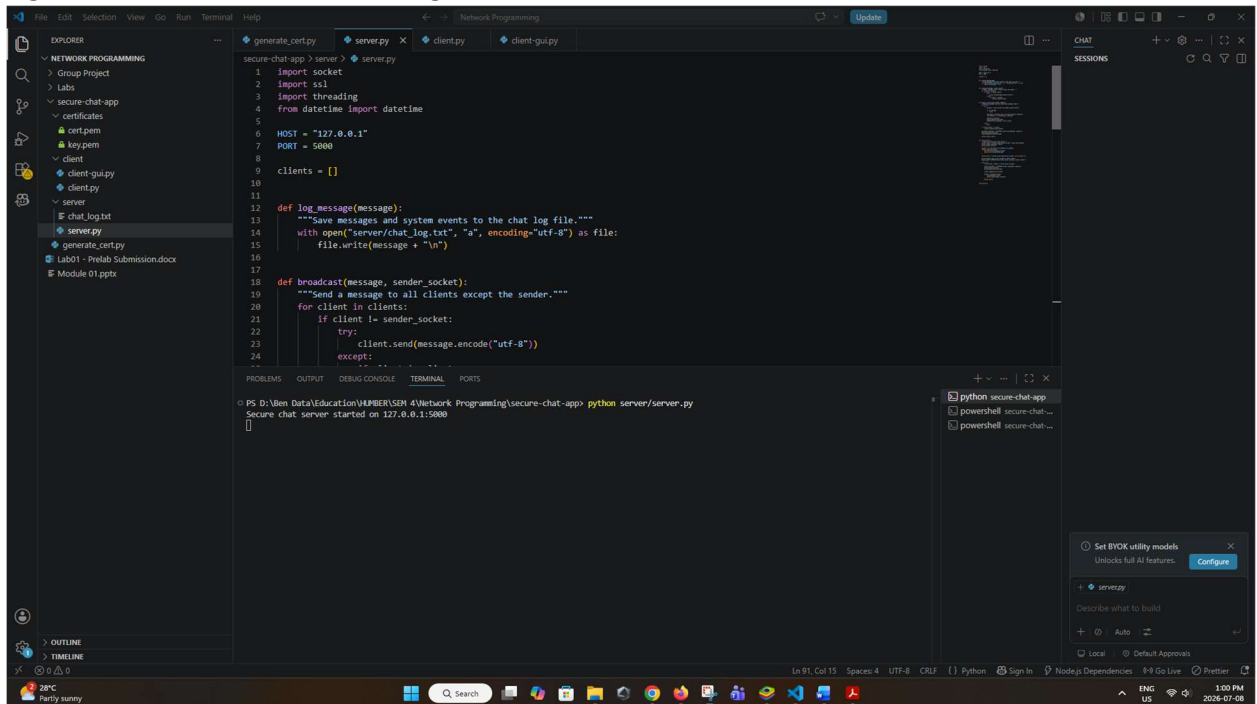
The application uses SSL/TLS encryption by wrapping TCP sockets with Python's ssl module. A self-signed certificate generated using the Cryptography library provides encrypted communication between the server and clients. Although certificate verification is disabled for development purposes, all transmitted data remains encrypted during network communication.

5. Application Workflow

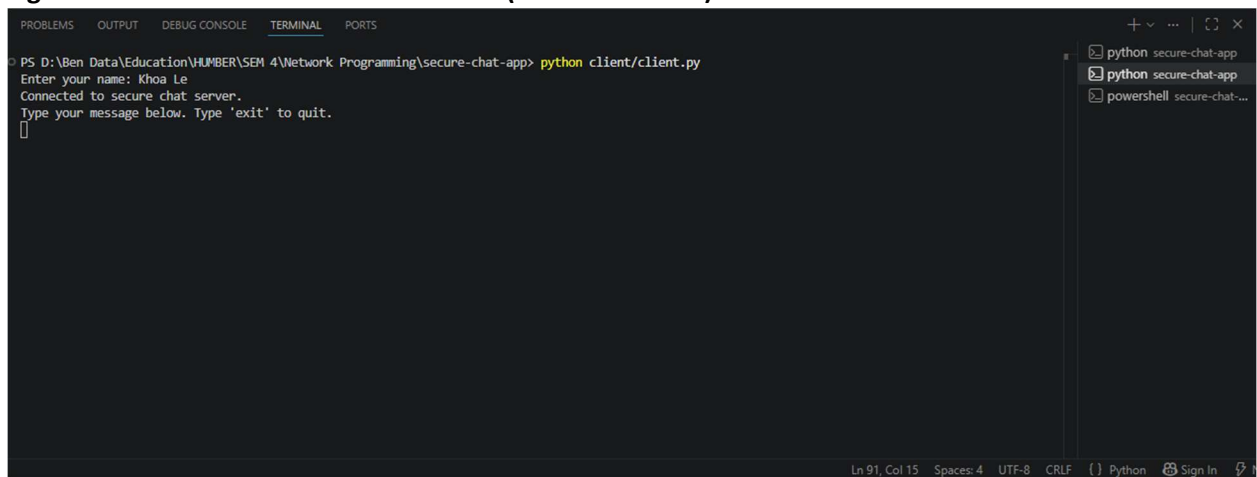


6. Results

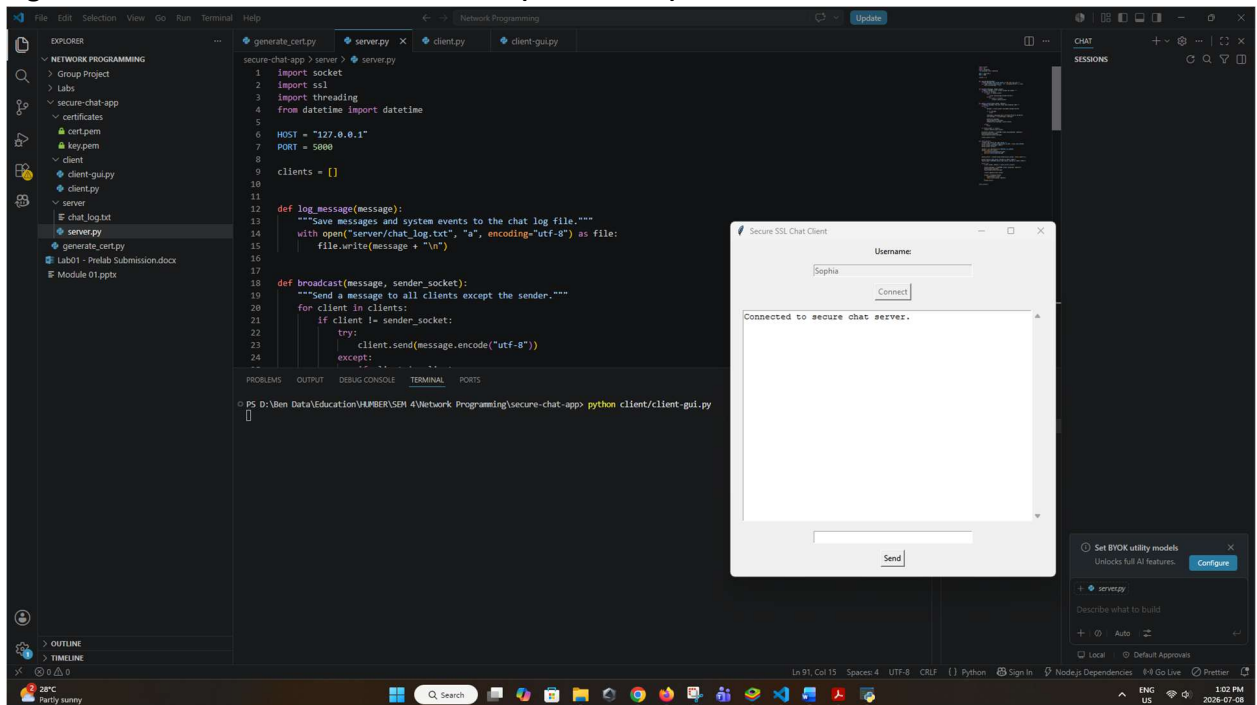
- **Figure 1: Secure chat server running.**



- **Figure 2a: Client connected to the server (console version).**

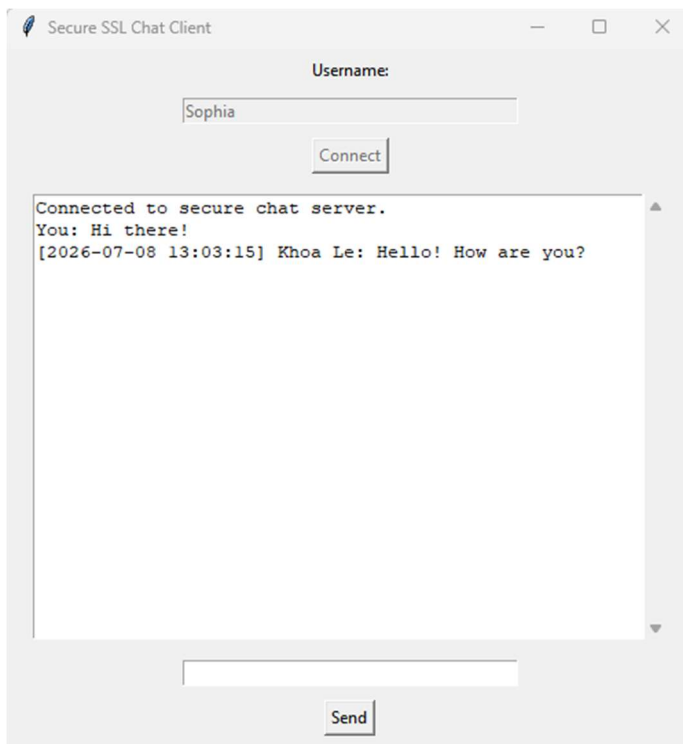


- **Figure 2b: Clients connected to the server (GUI version).**

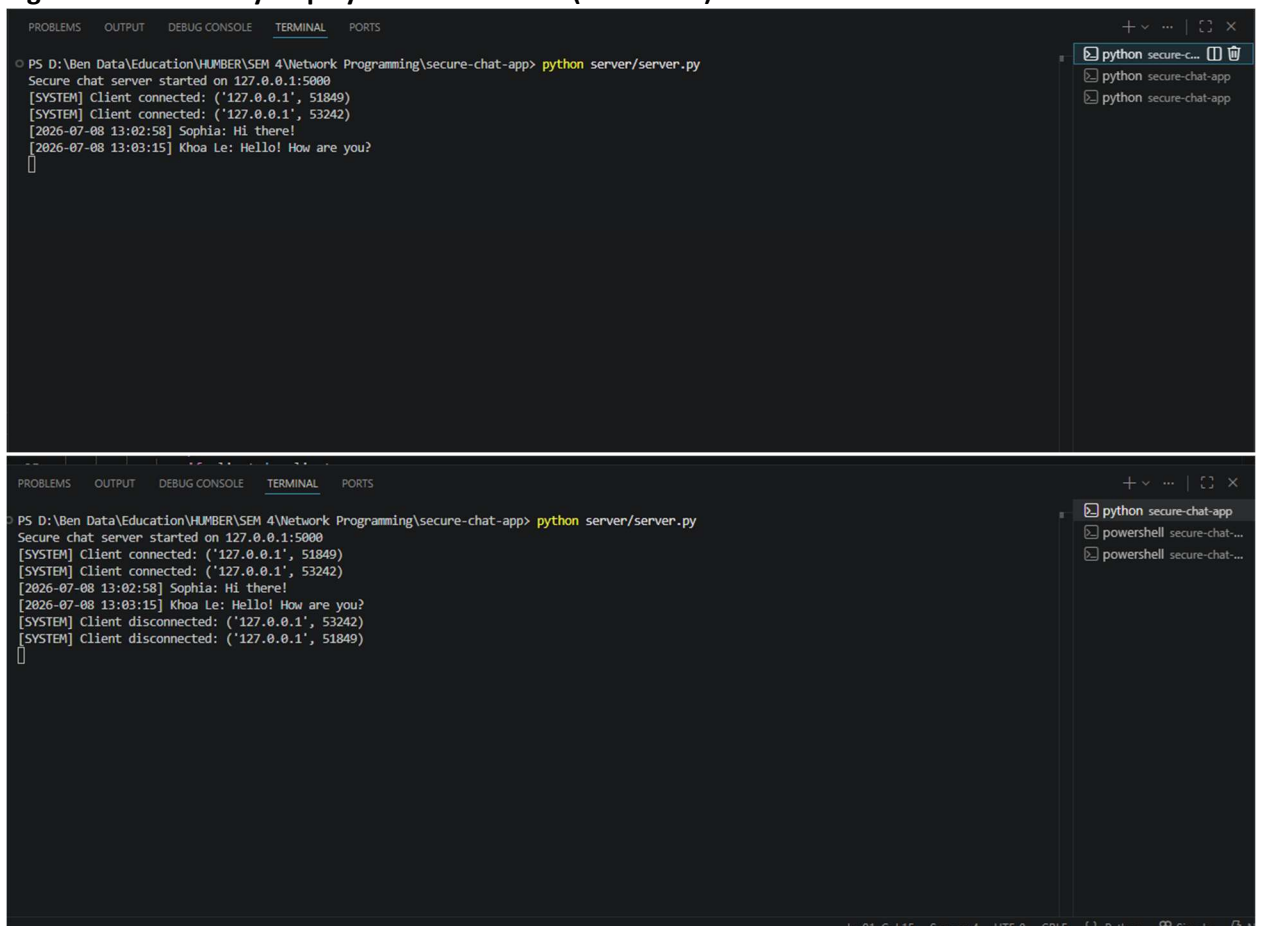


- **Figure 3: Two clients exchanging encrypted messages (screenshots from the clients both GUI and console version).**

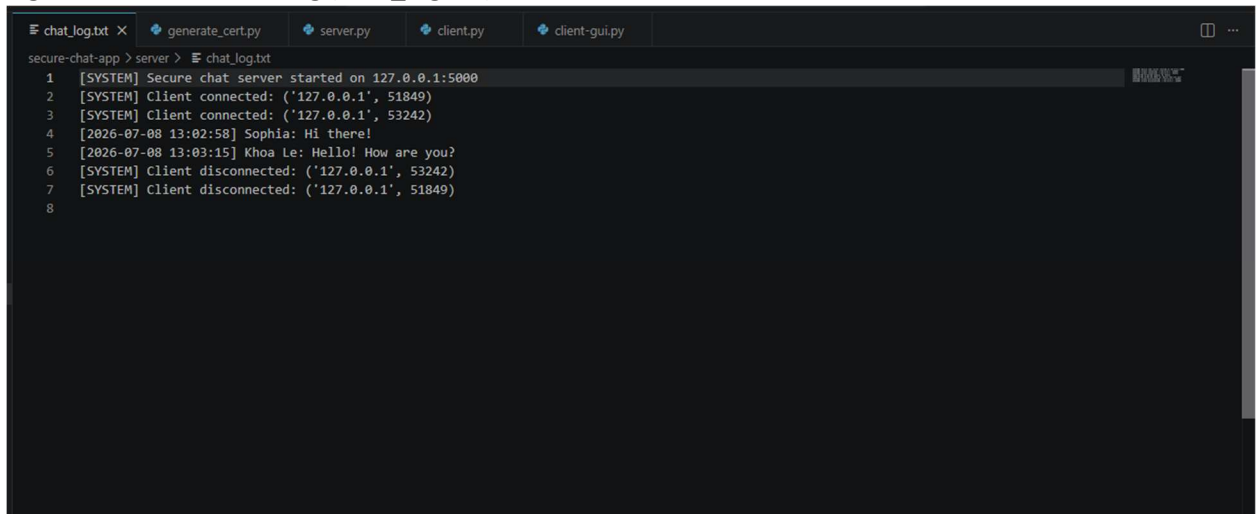




- **Figure 4: Chat history displayed in the console (the server)**



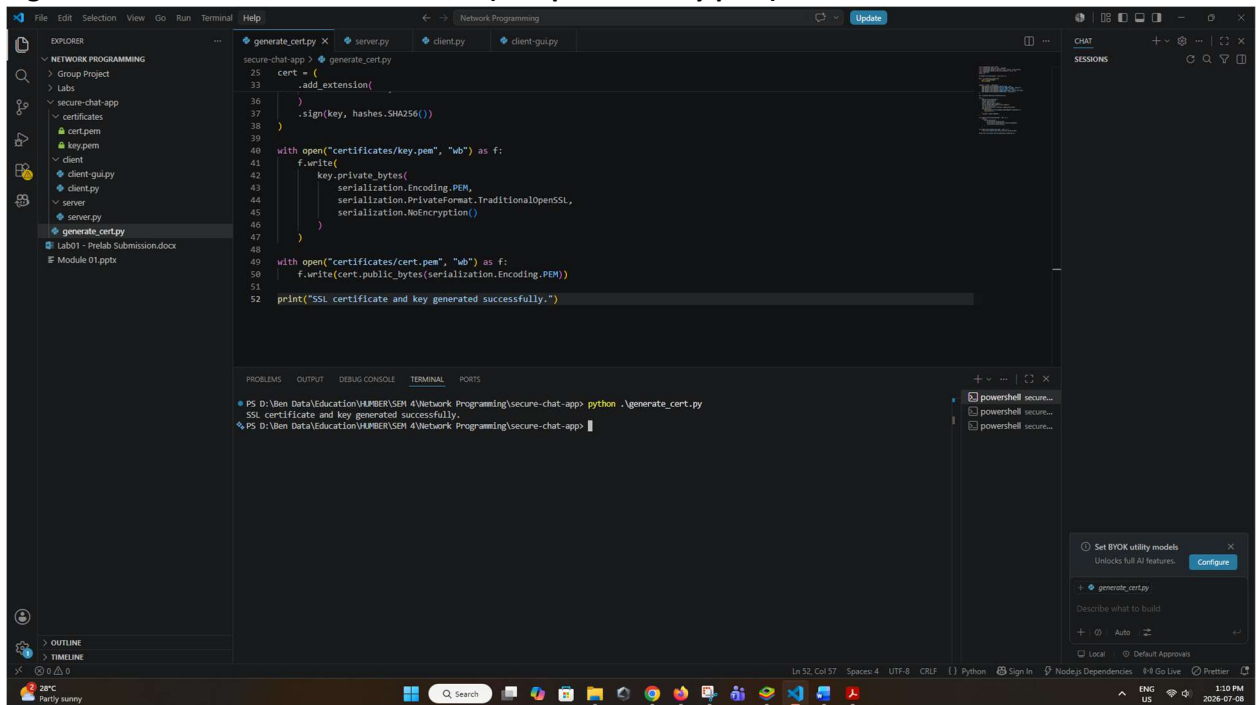
- **Figure 5: Conversation log (chat_log.txt).**



The screenshot shows a code editor with a file named `chat_log.txt` open. The editor has a dark theme and a sidebar on the left showing the file explorer. The chat log content is as follows:

```
secure-chat-app > server > chat_log.txt
1 [SYSTEM] Secure chat server started on 127.0.0.1:5000
2 [SYSTEM] Client connected: ('127.0.0.1', 51849)
3 [SYSTEM] Client connected: ('127.0.0.1', 53242)
4 [2026-07-08 13:02:58] Sophia: Hi there!
5 [2026-07-08 13:03:15] Khoa Le: Hello! How are you?
6 [SYSTEM] Client disconnected: ('127.0.0.1', 53242)
7 [SYSTEM] Client disconnected: ('127.0.0.1', 51849)
8
```

- **Figure 6: Generated SSL certificate files (cert.pem and key.pem).**



The screenshot shows a code editor with a file named `generate_cert.py` open. The editor has a dark theme and a sidebar on the left showing the file explorer. The code in `generate_cert.py` is as follows:

```
25 cert = {
26     .add_extension(
27         .sign(key, hashes.SHA256())
28     )
29 }
30
31 with open("certificates/key.pem", "wb") as f:
32     f.write(
33         key.private_bytes(
34             serialization.Encoding.PEM,
35             serialization.PrivateFormat.TraditionalOpenSSL,
36             serialization.NoEncryption()
37         )
38     )
39
40 with open("certificates/cert.pem", "wb") as f:
41     f.write(cert.public_bytes(serialization.Encoding.PEM))
42
43 print("SSL certificate and key generated successfully.")
```

The bottom of the screenshot shows a terminal window with the following output:

```
PS D:\Ben Data\Education\VM\BEN\SEM 4\Network Programming\secure-chat-app> python .\generate_cert.py
SSL certificate and key generated successfully.
PS D:\Ben Data\Education\VM\BEN\SEM 4\Network Programming\secure-chat-app>
```


7. Testing

Test Case	Expected Result	Actual Result	Status
Server starts successfully	Server begins listening	Successful	Pass
Client connects to server	Connection established	Successful	Pass
SSL handshake	Secure connection established	Successful	Pass
Send text message	Message delivered	Successful	Pass
Multiple clients connected	Broadcast to all clients	Successful	Pass
Chat logging	Messages stored in log file	Successful	Pass
Client disconnects	Server remains operational	Successful	Pass
GUI message display	Messages appear instantly	Successful	Pass

The application met all functional requirements during testing.

8. Limitations

Although the project satisfies all required objectives, several limitations remain:

- Uses a self-signed SSL certificate instead of a trusted Certificate Authority.
- Supports only local network communication.
- No user authentication or login system.
- Does not support private messaging.
- Does not allow file sharing.

9. Conclusion and Further Improvements

This project successfully developed a secure chat application using Python. By combining socket programming, SSL encryption, multithreading, and a graphical user interface, the application enables users to exchange encrypted messages in real time while maintaining a conversation history.

The project provided practical experience in secure network programming and demonstrated how SSL/TLS can be integrated into Python applications with minimal complexity.

Future improvements may include:

- User authentication and account management.
- SQLite or MySQL database integration.
- End-to-end encryption.
- File transfer support.
- Group chat functionality.
- Emoji and rich text support.
- Online deployment.
- Server administration GUI.

10. Reference

- Python Software Foundation. (2025). Python 3 Documentation. <https://docs.python.org/3/>
- Python Software Foundation. (2025). socket — Low-level networking interface. <https://docs.python.org/3/library/socket.html>
- Python Software Foundation. (2025). ssl — TLS/SSL wrapper for socket objects. <https://docs.python.org/3/library/ssl.html>
- Python Software Foundation. (2025). threading — Thread-based parallelism. <https://docs.python.org/3/library/threading.html>
- Python Software Foundation. (2025). Tkinter Documentation. <https://docs.python.org/3/library/tkinter.html>
- The Cryptography Developers. (2025). Cryptography Documentation. <https://cryptography.io/>